南京郵電大學
Nanjing University of Posts and Telecommunications

Data Structures

Lecture 10: Introduction to Graphs

Prof. Wang Haiyan
School of Computer Science
wanghy@njupt.edu.cn

Today's Topic

For the next three lectures, we turn our attention to graphs

Today:

- The language of graphs
- Why graphs are useful
- Graphs as data structures: how should we represent a graph using concrete structures?

Today's Topic

All of the abstract data structures we've looked at so far store individual elements

We don't care about how those elements are related (except perhaps whether they have smaller or larger keys)

Graphs allow us to capture **relationships** between elements

This is extremely powerful:

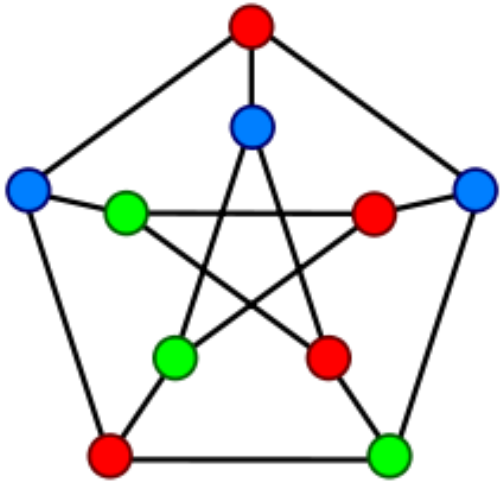
- Road/Social/Computer networks
- Protein structures
- Electronic circuits
- Academic paper citations
- Relationship between features in an image
- 3D meshes

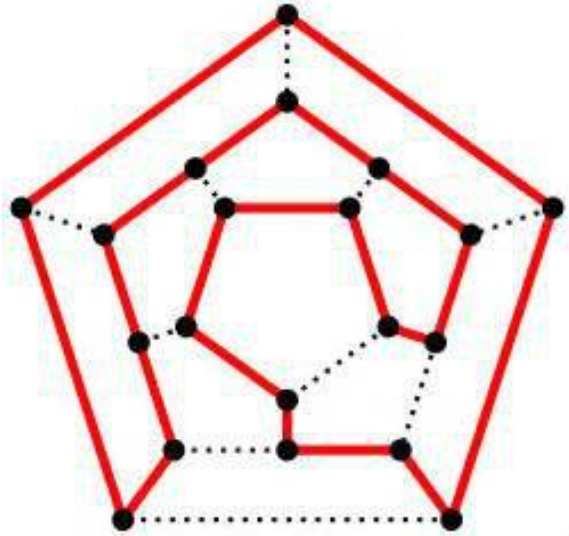
Graph Applications

There are a huge number of applications of graphs in algorithm design

Graphs are everywhere in computer science!

One of the unifying themes



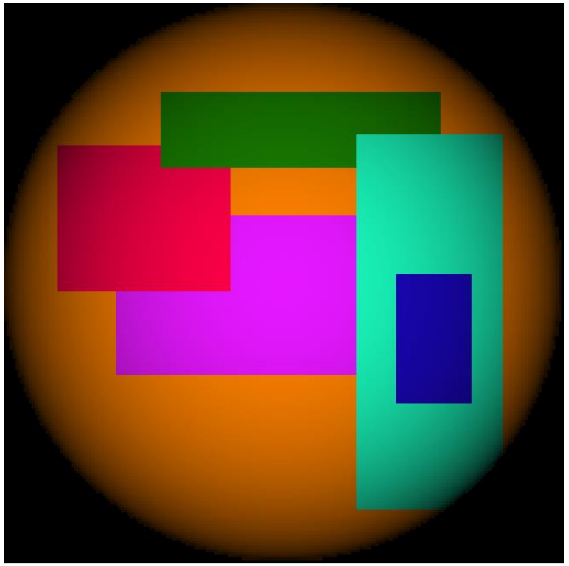


Graph Applications: Road Networks

Ctrip, tuniu, google maps, AA route planner etc are nothing more than a toolbox of graph algorithms which can be applied to a road network graph

Efficiently finding the fastest route between two points is the key tool, subject to constraints such as “avoid motorways” or “provide a route suitable for cycling/walking”

Graph Applications: Image Segmentation



Input image

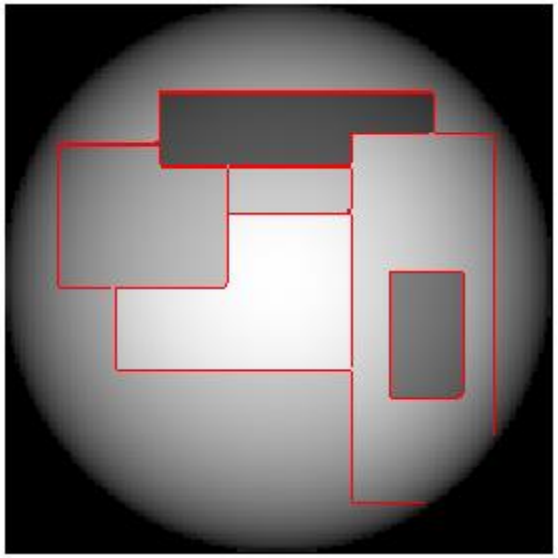
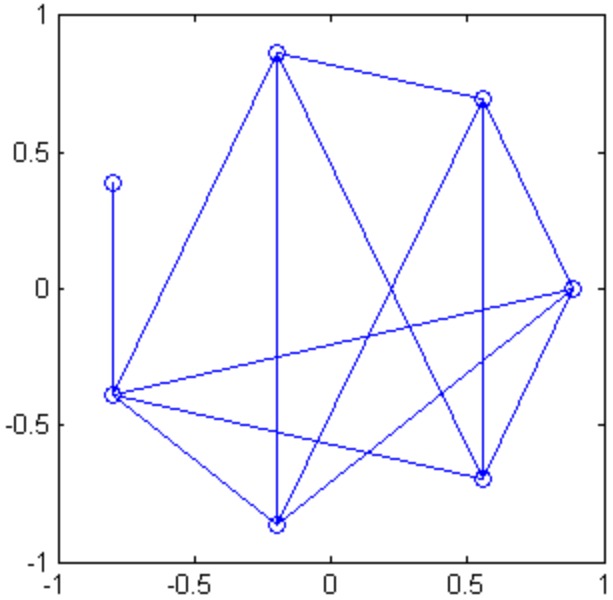


Image segmented by colour

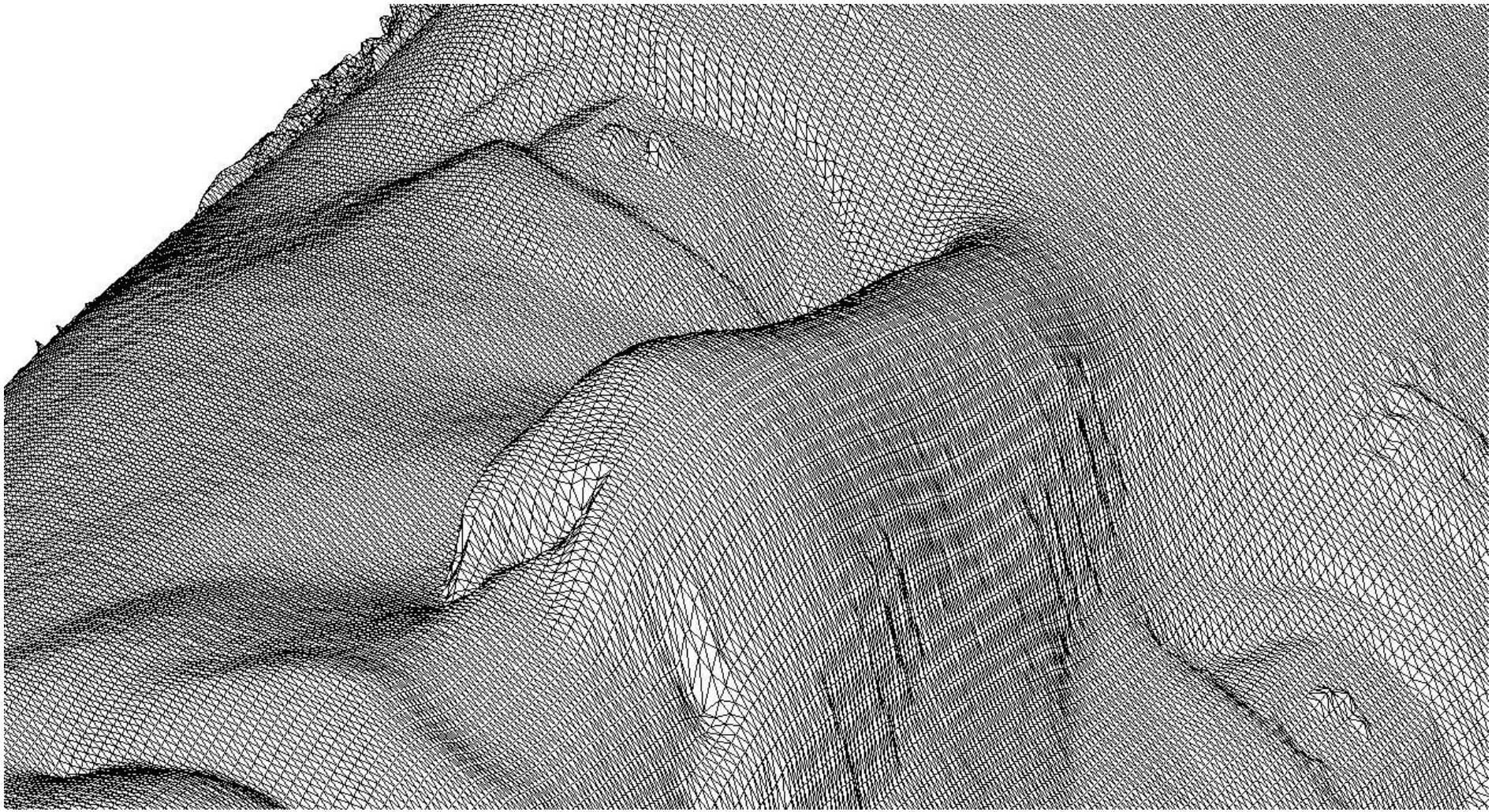


Graph: regions are nodes, edge means regions share a boundary

We can compute the relative reflectivity of each region

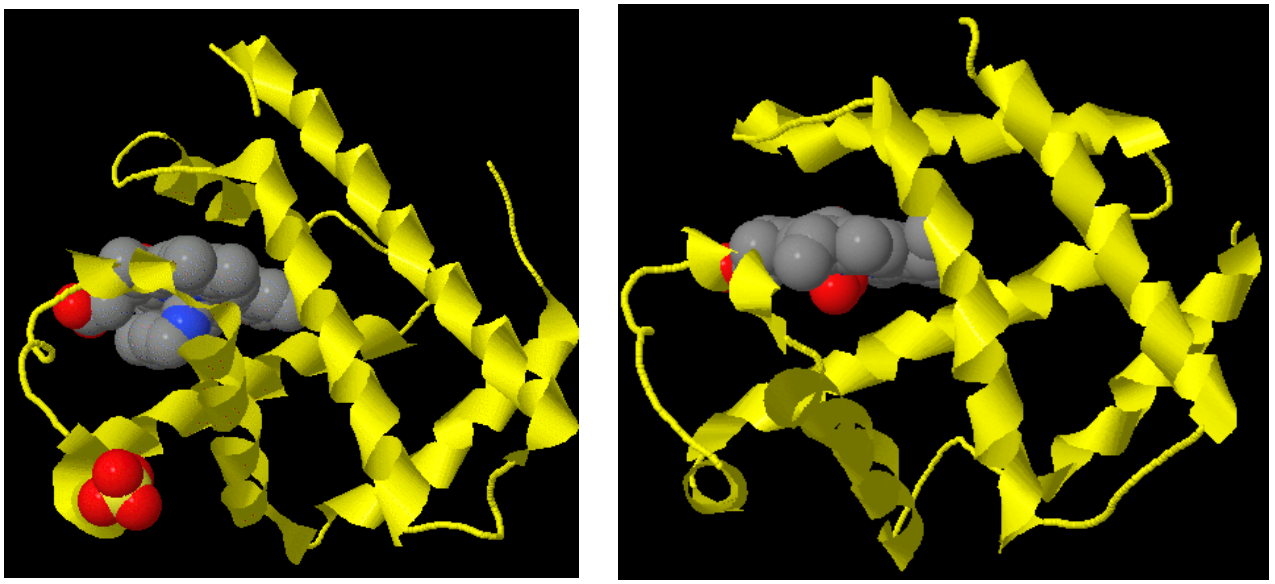
Traversing the graph gives us a way to assign absolute values

Graph Applications: 3D meshes



Mesh simplification, 3D object recognition, geodesic paths etc

Graph Applications: Protein Structures/DNA



Graph algorithms are being used to help design new drugs to treat diseases, help understand genetic structure etc

Graph Applications: Social Networks

Websites such as facebook now have huge graphs representing social networks

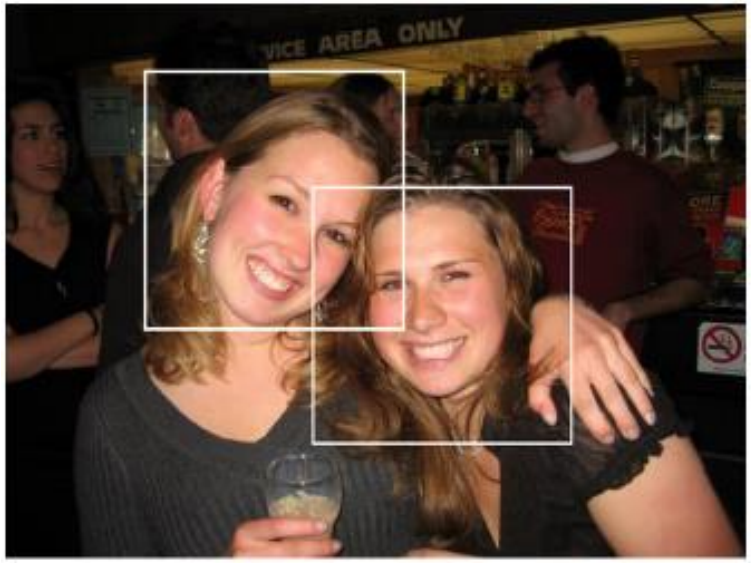
This can be used to ask questions like:

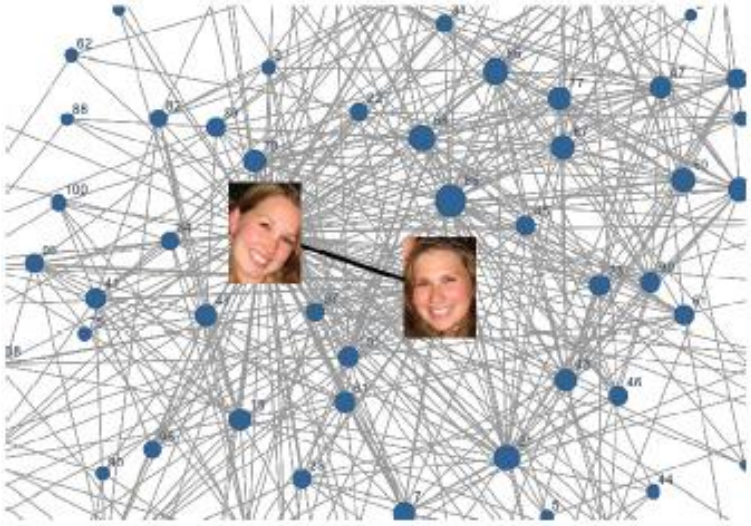
Who is this person likely to know?

What is the shortest path from person A to B?

What is the minimum number of connections between any two people? (6 degrees of separation etc)

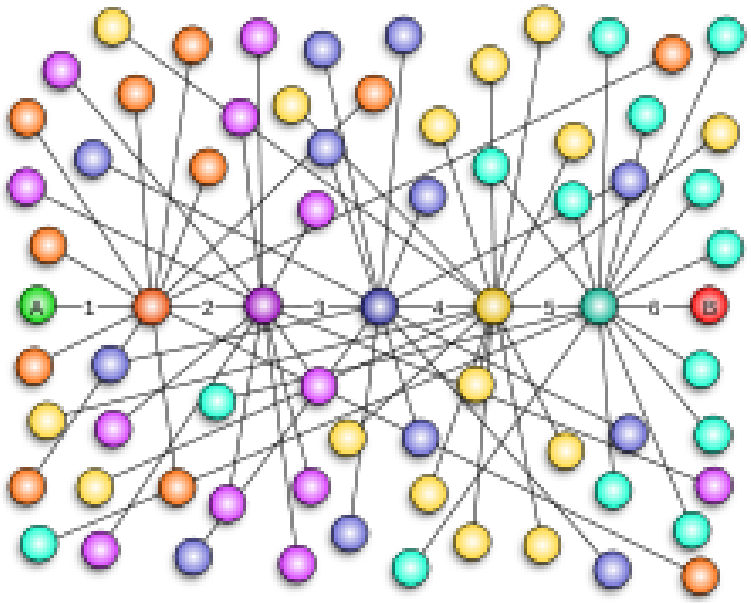
Or even: I've identified person A in this image, who are likely candidates for the identity of person B?





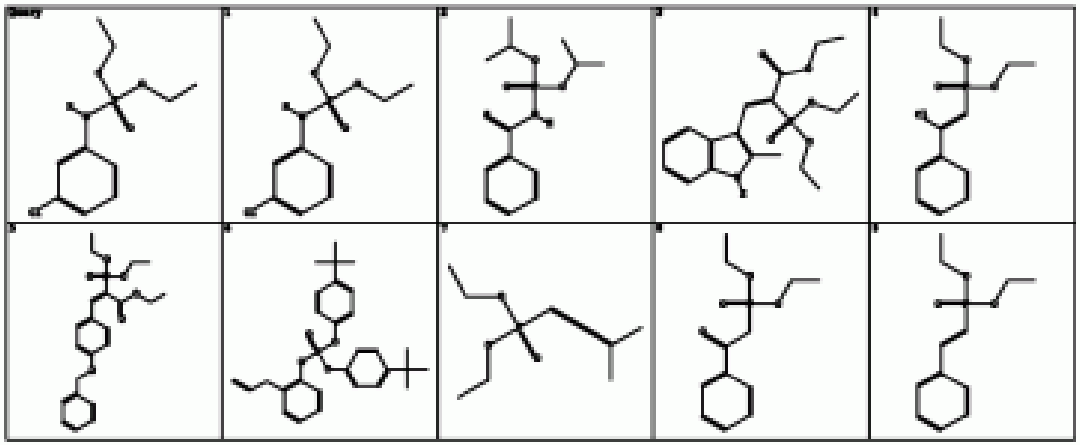
Graph Applications: Social Networks

Six degrees of separation is the idea that all living things and everything else in the world are six or fewer steps away from each other so that a chain of "a friend of a friend" statements can be made to connect any two people in a maximum of six steps.



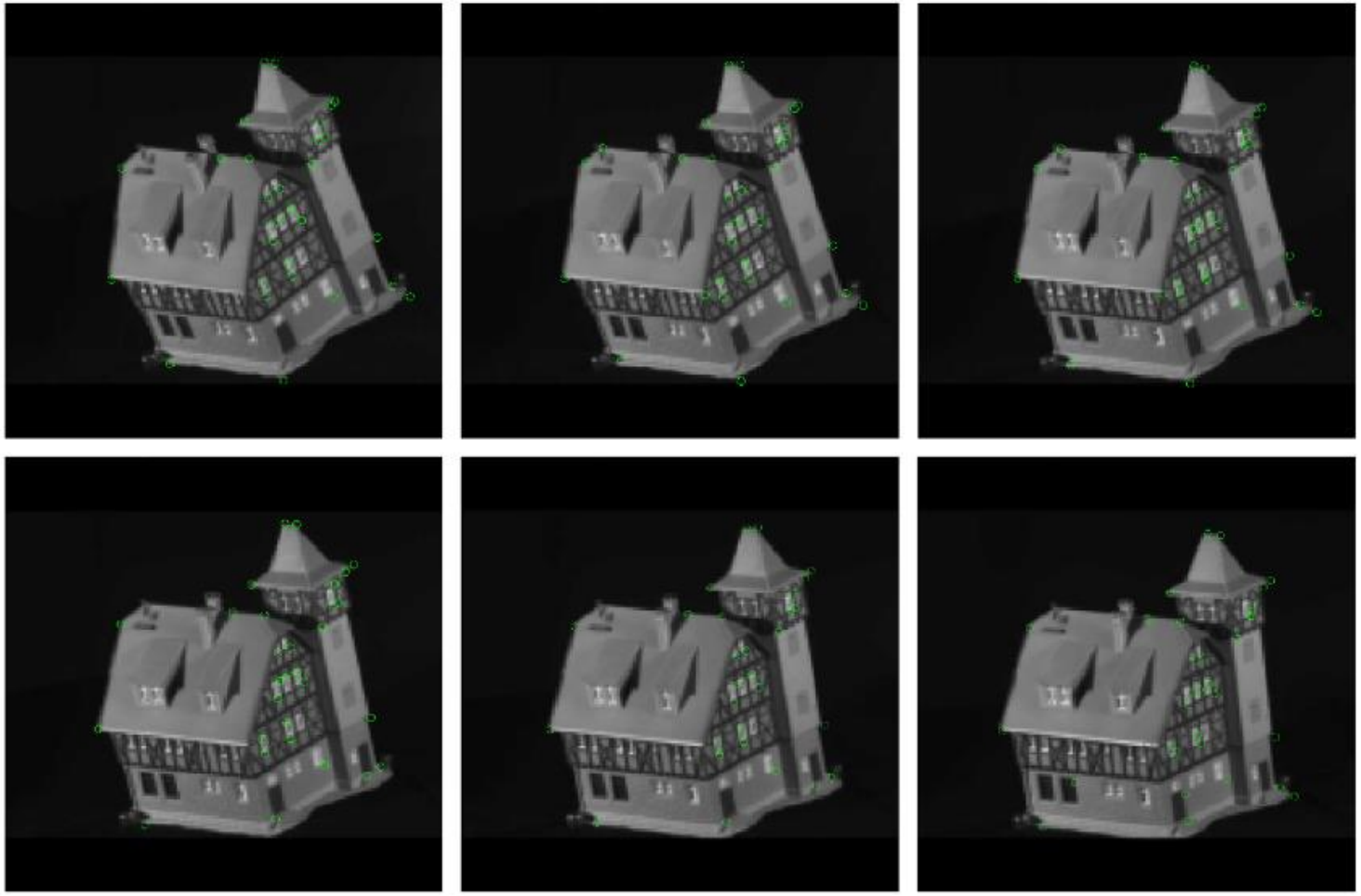
Trypophobia

Graph Applications: Chemical Structures



Atom bond graphs

Graph Applications: Object Recognition



Arrangement of features within an image – recognition across changing viewpoint by matching graphs

Graphs

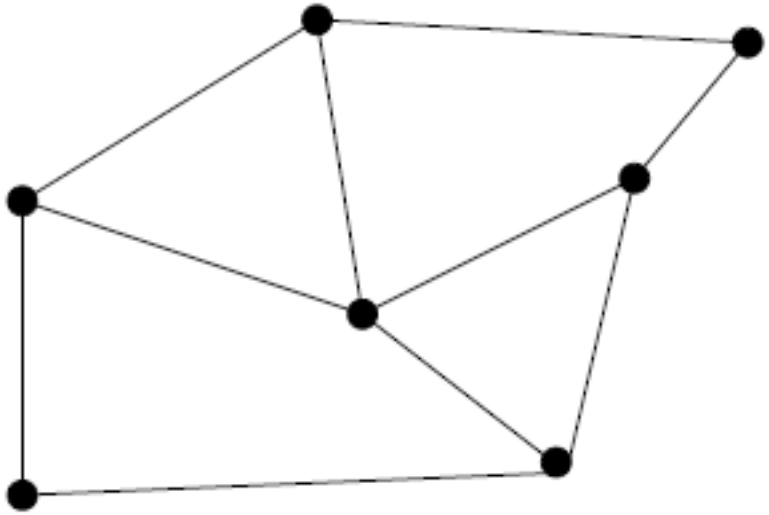
A graph G = (V,E) is defined by:

- A set of ***vertices*** or ***nodes*** V
- A set of ***edges*** E, consisting of ordered or unordered pairs of vertices from V

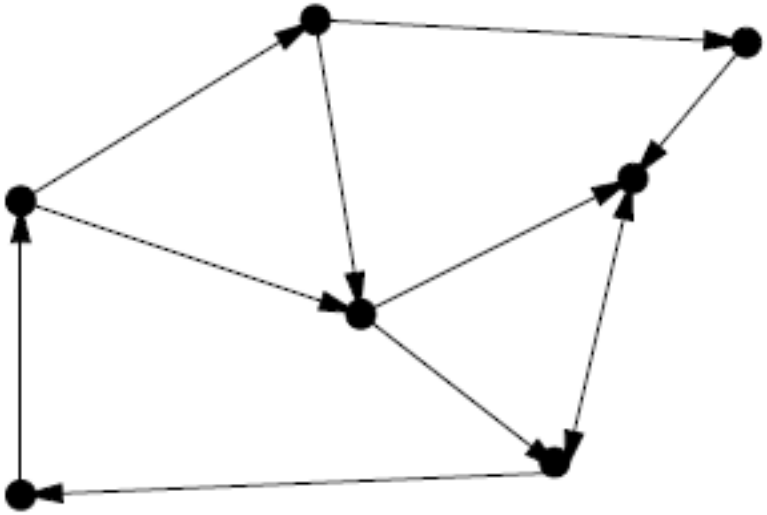
Graphs come in many flavours…

Undirected versus Directed

A graph is ***undirected*** if an edge (x,y) is a member of E, then (y,x) is also a member of E: $(x, y) \in E \Rightarrow (y, x) \in E$



Undirected



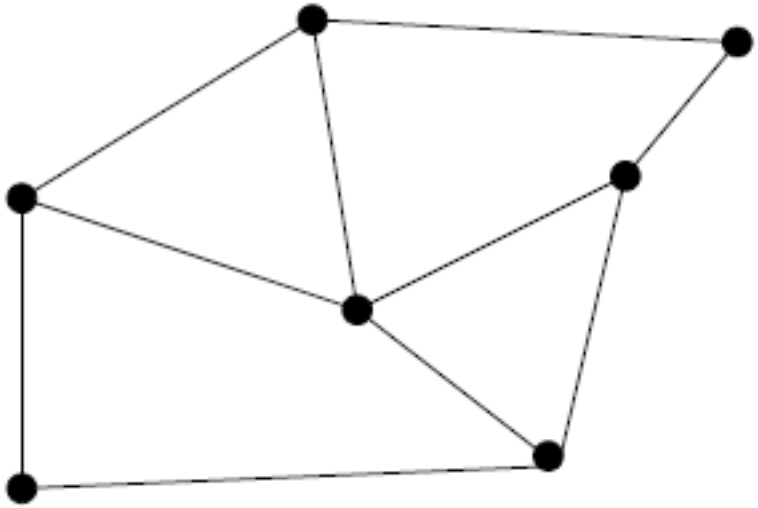
Directed

Road networks are typically directed (one-way streets)

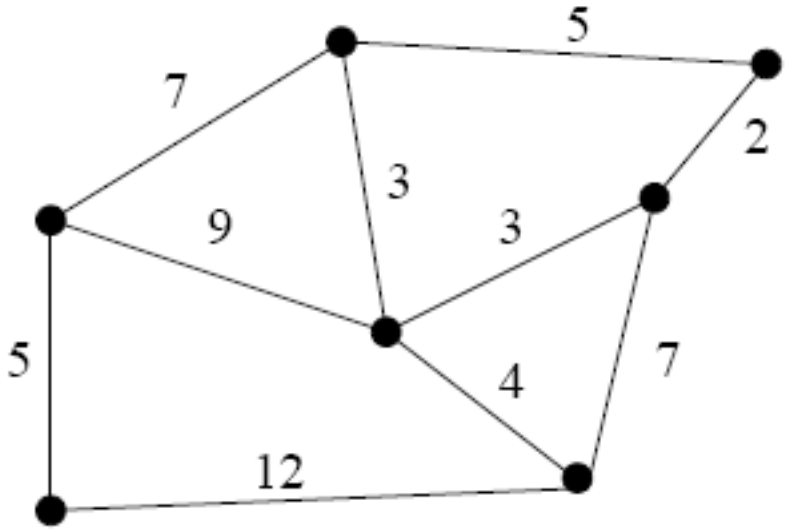
Social network graphs where an edge indicates friendship are (hopefully!) undirected

Unweighted versus Weighted

In a ***weighted graph*** each edge is assigned a numerical weight:



Unweighted

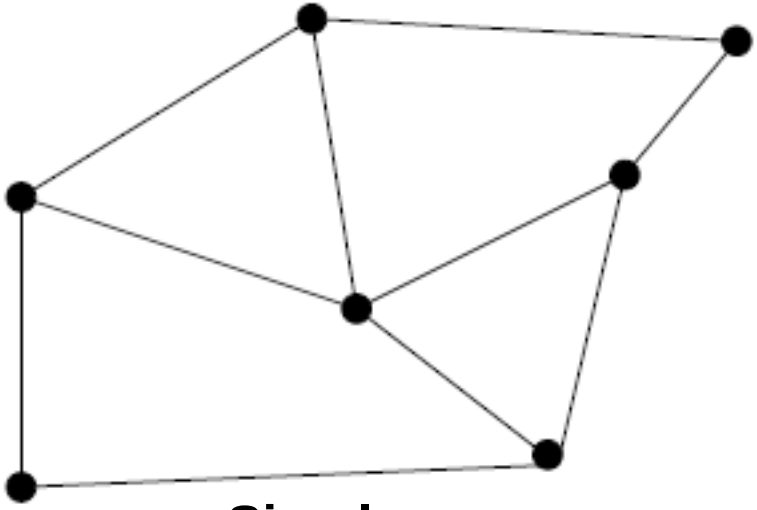


Weighted

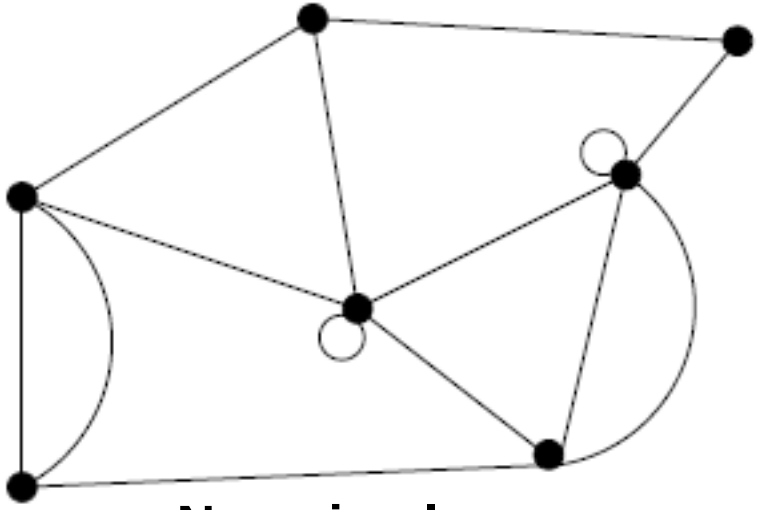
For a road network graph, the edge cost may correspond to distance or time taken to drive

Simple versus Non-simple

When we design algorithms to process graphs, certain types of edges complicate matters. For example: a ***self-loop*** is an edge (x,x) which connects a vertex to itself, a ***multi-edge*** is an edge which occurs more than once in a graph. A simple graph does not contain these structures:



Simple

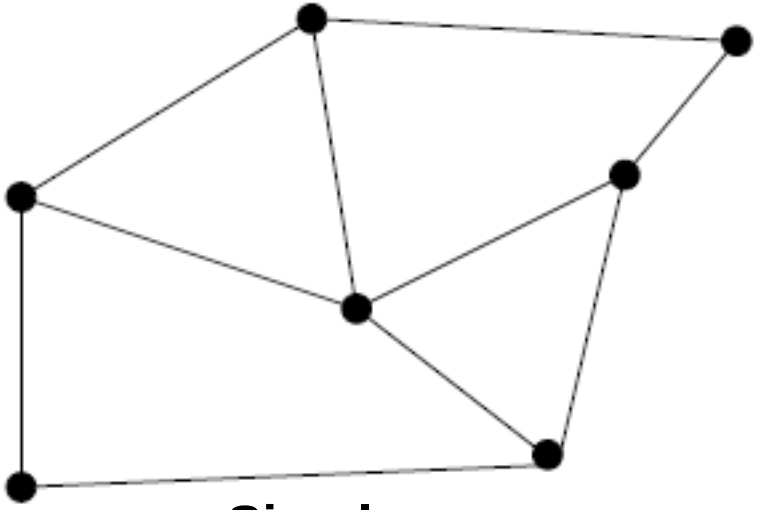


Non-simple

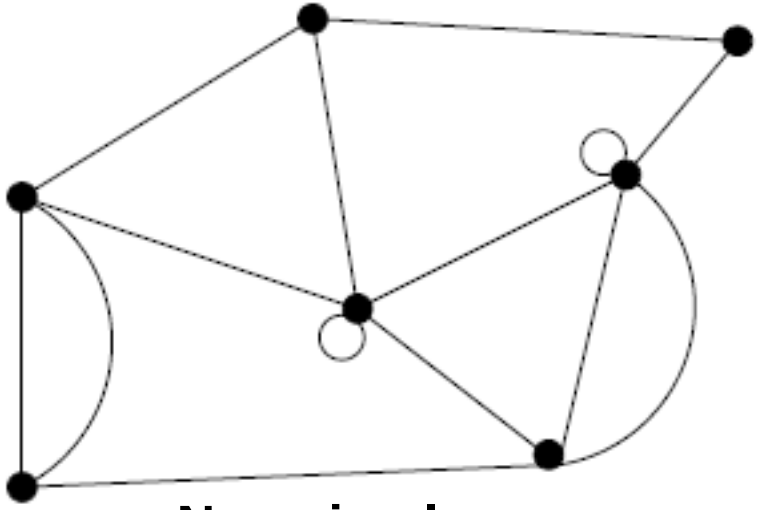
Q: If we represent E as a set, can we describe multi-edges? If not, what is the right structure to use?

Simple versus Non-simple

When we design algorithms to process graphs, certain types of edges complicate matters. For example: a ***self-loop*** is an edge (x,x) which connects a vertex to itself, a ***multi-edge*** is an edge which occurs more than once in a graph. A simple graph does not contain these structures:



Simple



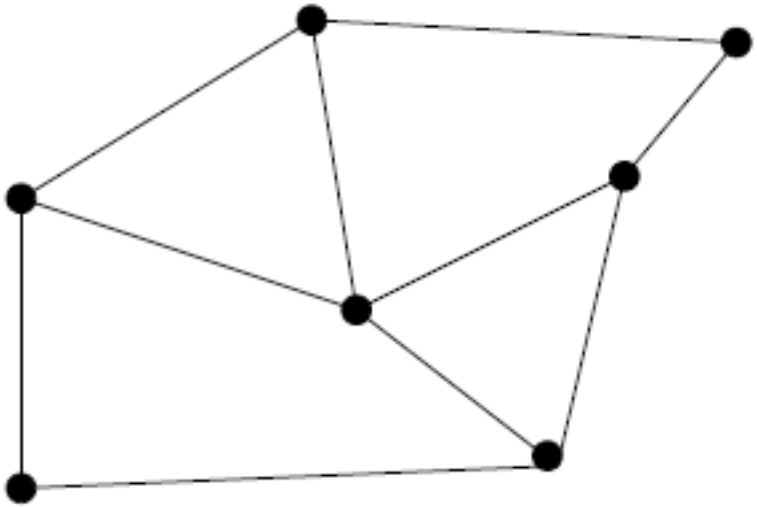
Non-simple

Q: If we represent E as a set, can we describe multi-edges? If not, what is the right structure to use?

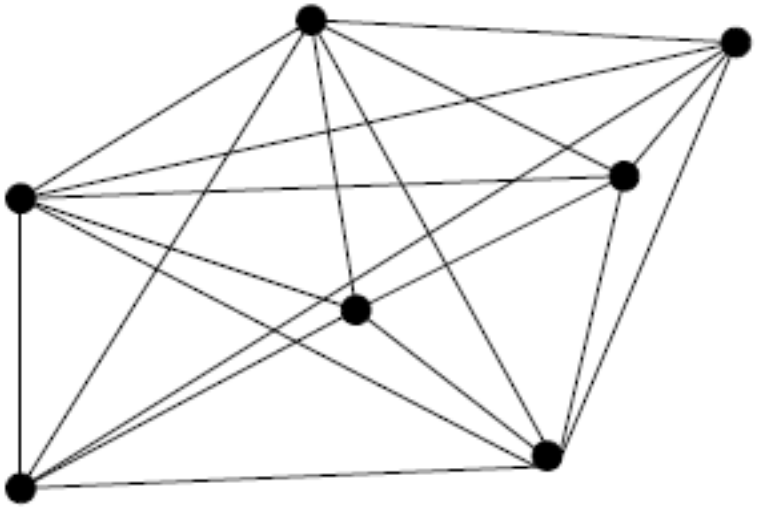
A: A bag or multiset

Sparse versus Dense

A ***sparse graph*** is one in which only a small fraction of all the vertex pairs are connected by an edge:



Sparse



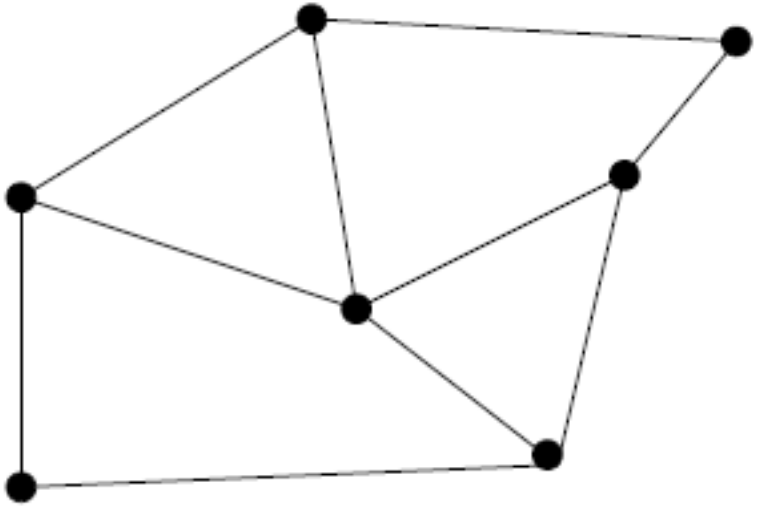
Dense

The number of edges in a sparse graph will grow approximately linearly with the number of vertices. In a complete graph, every vertex pair is connected by an edge.

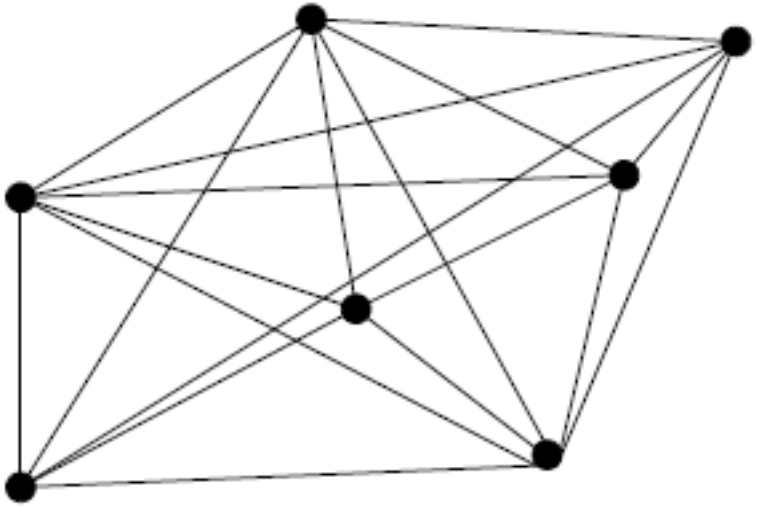
Q: How many edges in a complete graph?

Sparse versus Dense

A ***sparse graph*** is one in which only a small fraction of all the vertex pairs are connected by an edge:



Sparse



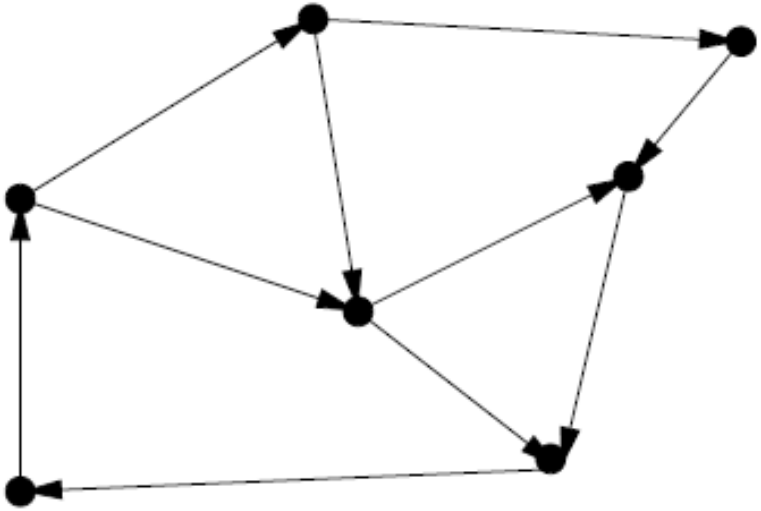
Dense

The number of edges in a sparse graph will grow approximately linearly with the number of vertices. In a complete graph, every vertex pair is connected by an edge.

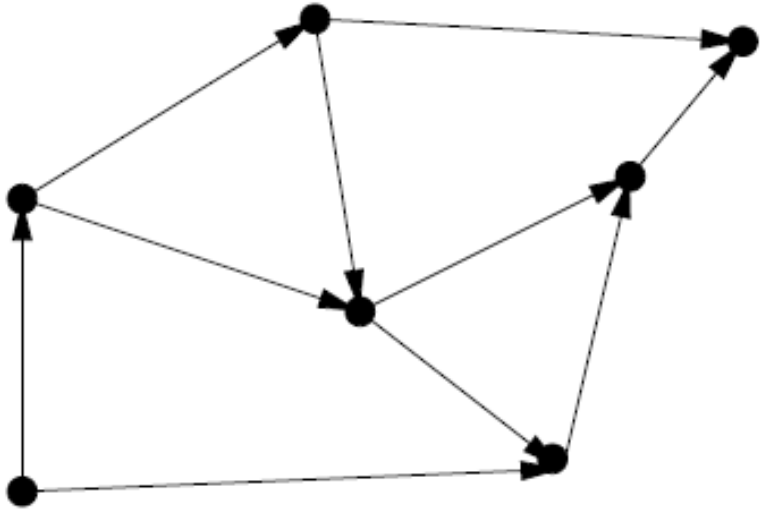
Q: How many edges in a complete graph? **A**: n vertices = n(n-1)/2 edges

Cyclic versus Acyclic

An ***acyclic graph*** is one which does not contain any cycles:



Cyclic

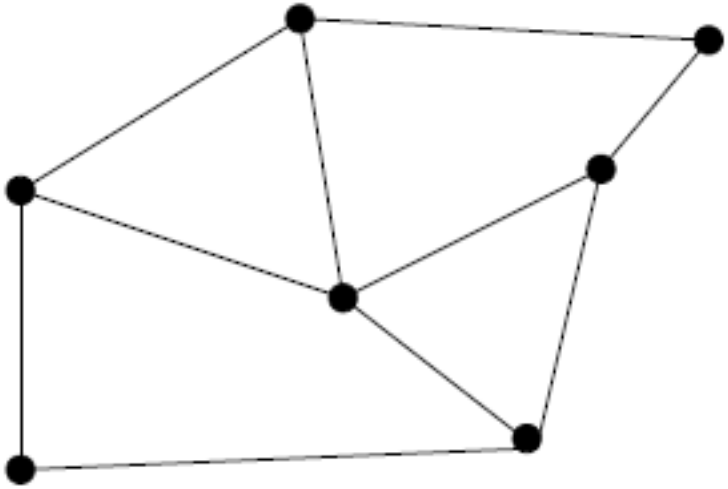


Acyclic

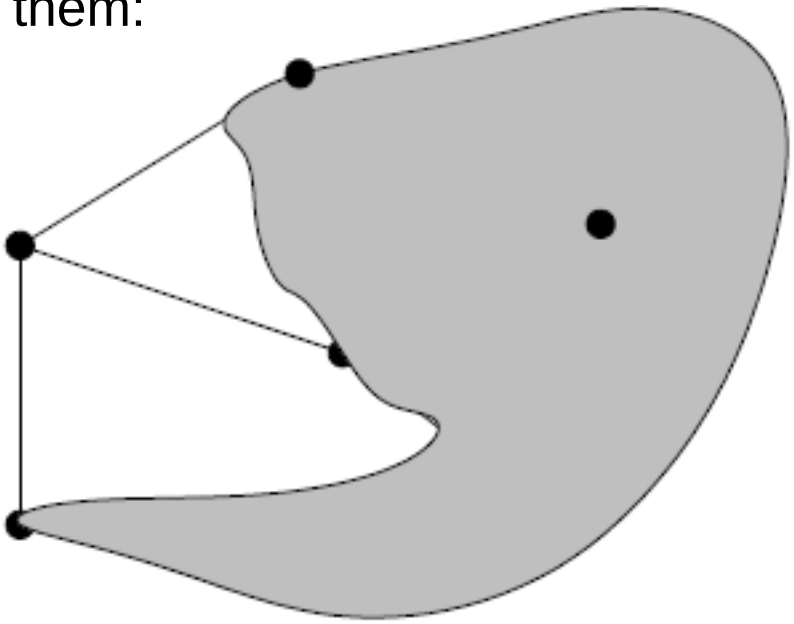
A common flavour of graph is a ***directed acyclic graph*** (DAG). Scheduling problems are described using DAGs, such that a directed edge (x,y) indicates that x occurs before y.

Explicit versus Implicit

For many algorithms, we may not explicitly store the graph but only construct vertices explicitly when we visit them:



Explicit

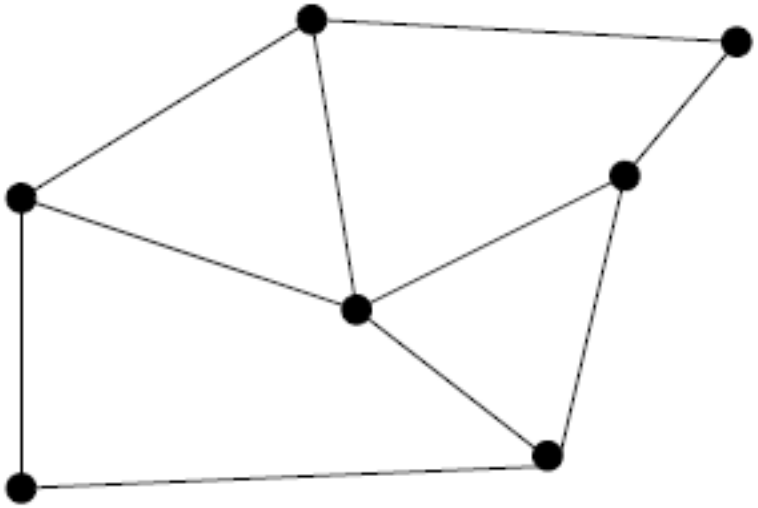


Implicit

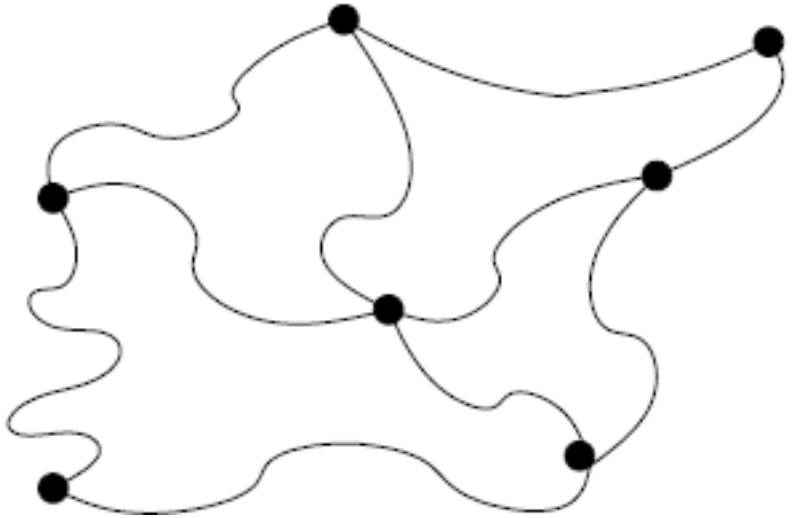
This is the case in backtracking search, for example when we search a graph of game states. In chess, we do not explicitly store the whole graph of game states but rather construct a portion of it connected to the current state.

Embedded versus Topological

A graph is ***embedded*** if the vertices and edges have geometric meaning. Without an embedding, we are concerned only with the ***toplogy*** of the graph:



Embedded

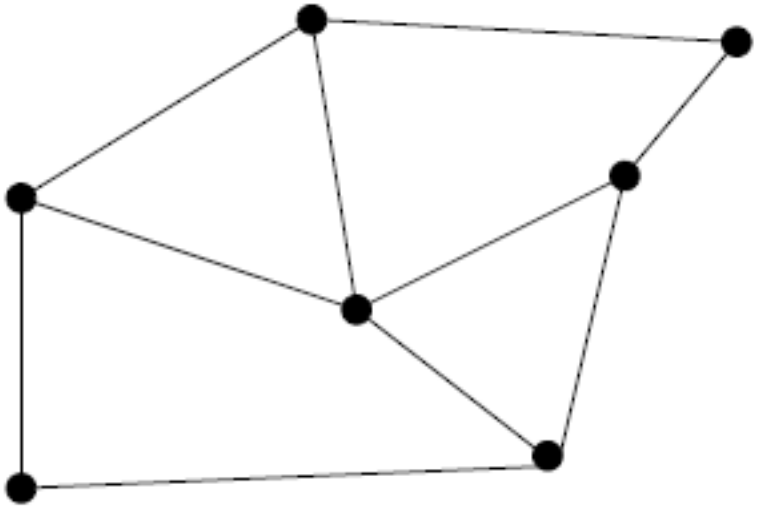


Topological

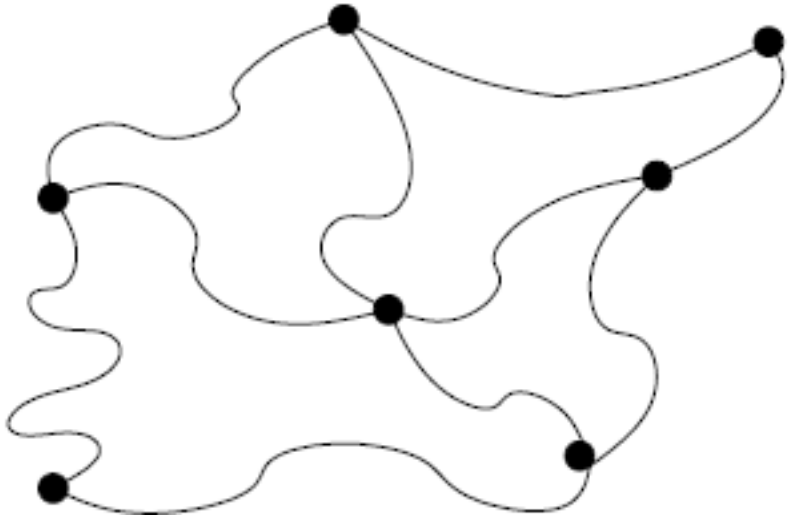
Q: Examples of embedded and topological graphs?

Embedded versus Topological

A graph is ***embedded*** if the vertices and edges have geometric meaning. Without an embedding, we are concerned only with the ***toplogy*** of the graph:



Embedded



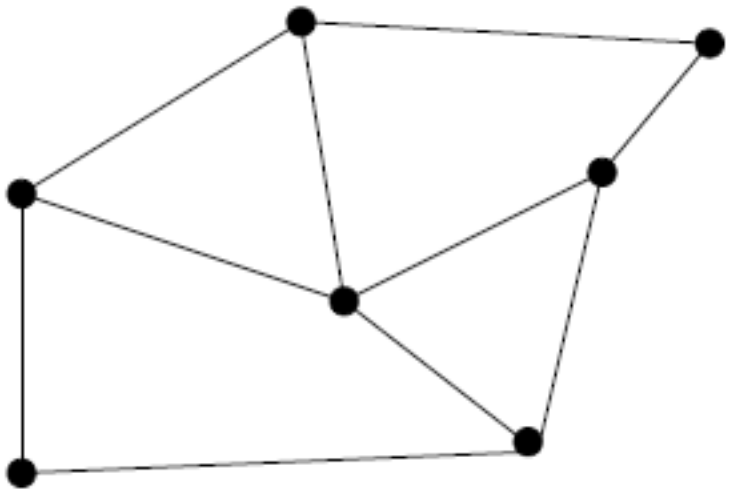
Topological

Q: Examples of embedded and topological graphs?

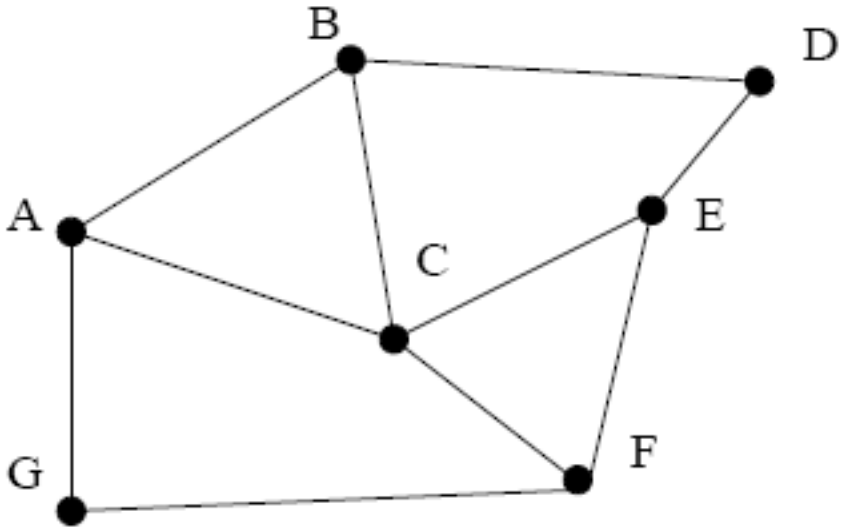
A: e.g. embedded – road network, circuit board, e.g. topological – social network

Unlabeled versus Labeled

A ***labeled graph*** assigns a unique name to each vertex:



Unlabeled



Labeled

The ***graph isomorphism*** problem is to determine whether two unlabeled graphs are identical (in other words we don't know which vertices are supposed to correspond with each other). This is a hard problem.

Graphs Operations

Before we think about how to implement a graph, what operations might we want to perform?

- **Accessing information**: given a node, extract information such as distance to another node or edge weights
- **Navigation**: given a node, access its outgoing edges or maybe its incoming edges
- **Edge queries**: given a pair of nodes determine whether there is an edge connecting them
- **Construction/conversion**: given data, build a graph or given one representation of a graph, convert it to another
- **Update**: add or remove edges/nodes

Representing Graphs

As we know, any abstract data type must be represented in terms of concrete structures

Two common structures for graphs:

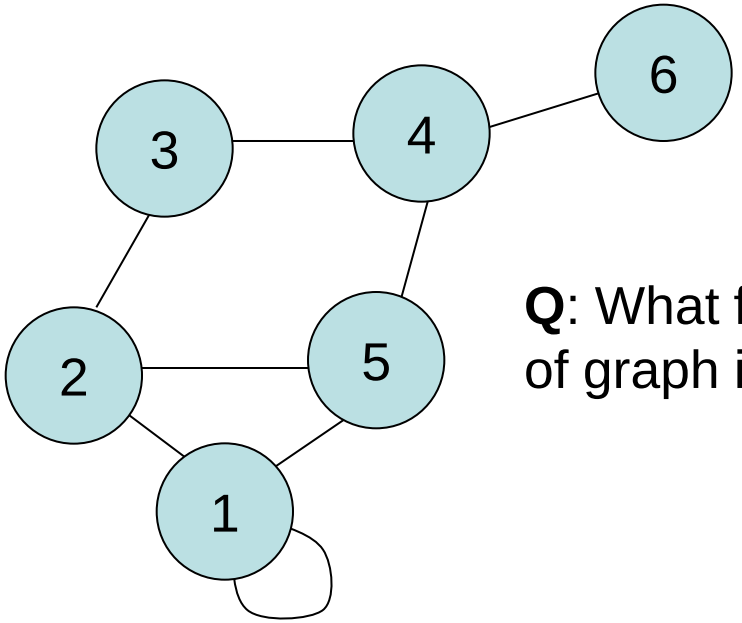
- Adjacency matrix
- Adjacency list

Best choice depends on the operations we want to do efficiently

Representing Graphs: Adjacency Matrix

Given a graph G = (V,E) containing n nodes and m edges, we can store the graph is an n by n matrix M

We set $M[i, j] = 1$ if (i,j) is an edge of G and 0 otherwise:



Q: What flavour of graph is this?

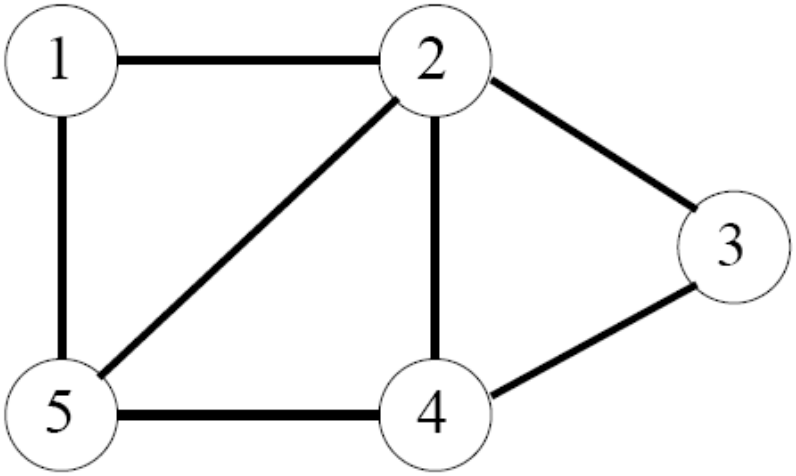
$$\begin{pmatrix} 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

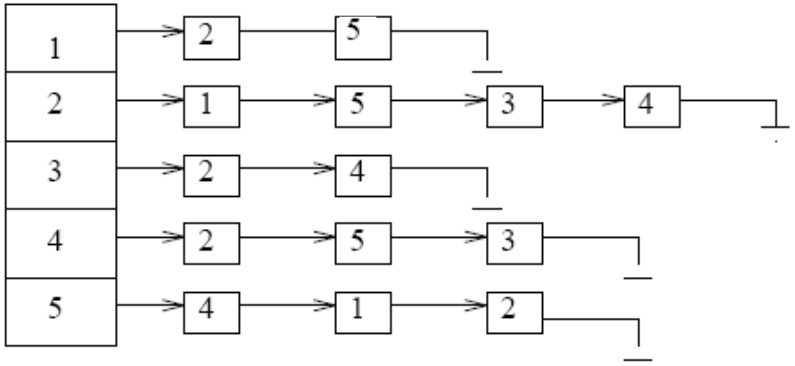
Q: What is inefficient about this representation? For what sort of graphs will it be particularly inefficient? Any way to improve this?

Representing Graphs: Adjacency List

Given a graph G = (V,E) containing n nodes and m edges, we can store the graph is an n-element array of pointers

The ith element points to a linked list of edges incident on node i:





To test if an edge (i,j) is in the graph, we traverse the ith linked list, looking for j. This is $O(d_i)$, where d_i is the degree of the ith node.

Comparison between representations

Comparison	Winner
Faster to test if (x, y) exists?	matrices
Faster to find vertex degree?	lists
Less memory on small graphs?	lists $(m + n)$ vs. (n^2)
Less memory on big graphs?	matrices (small win)
Edge insertion or deletion?	matrices $O(1)$
Faster to traverse the graph?	lists $m + n$ vs. n^2
Better for most problems?	lists

In addition, adjacency matrix can be used to perform graph operations using linear algebra

Both representations are useful depending on application

Adjacency lists tend to be more flexible as they use a linked structure

Conclusions

We should now know:

- What a graph is and why they may be useful
- Terminology to describe different flavours of graphs
- Representing graphs using concrete data structures

Next time:

- Exploring graphs: graph traversal

Recommended reading for this lecture:

- Mehlhorn Chapter 8
- Cormen Chapter 22.1
- Skiena 5.1 and 5.2